

Stack

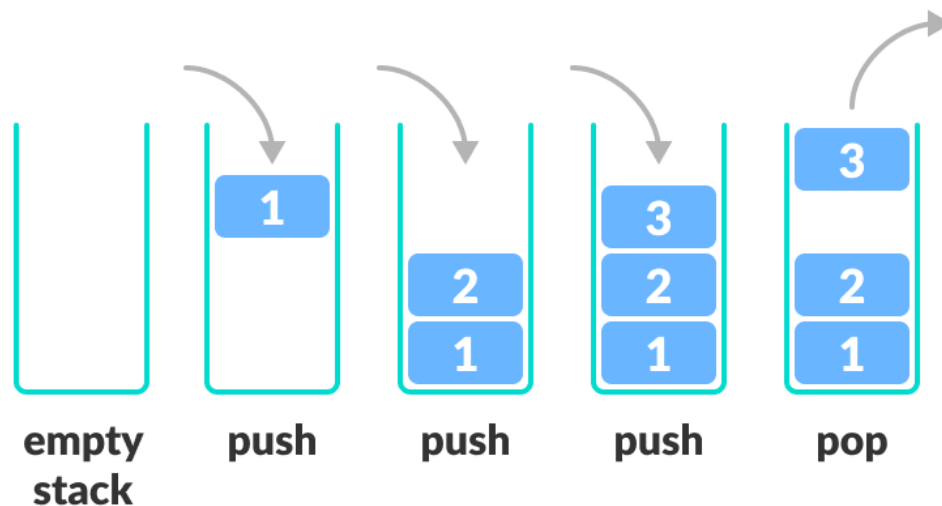
Unit-3

Stack is a linear data structure in which the insertion and deletion operations are performed at only one end. In a stack, adding and removing of elements are performed at a single position which is known as "top". That means, a new element is added at top of the stack and an element is removed from the top of the stack. In stack, the insertion and deletion operations are performed based on LIFO (Last In First Out) principle.

Stack ADT

Stack is a data structure in which elements can be added and removed from one end known as "TOP". The operations in Stack are performed in LIFO order. The last element inserted in the stack is the first element to be removed from the stack. Stack can be implemented using array or linked list. Array implementation of Stack is static in nature; that is the size of stack is fixed. Linked List implementation of Stack is dynamic.

In a linked list implementation, the stack is stored as a linked list of nodes. Each node in the linked list contains a value and a pointer to the next node. The top of the stack is always the first node in the linked list. When an element is pushed onto the stack, a new node is created and added to the beginning of the linked list. When an element is popped off the stack, the first node in the linked list is removed.



Applications

- Used in function call
- Conversion of notation expression
- Parenthesis matching
- Recursion

- Web browser history

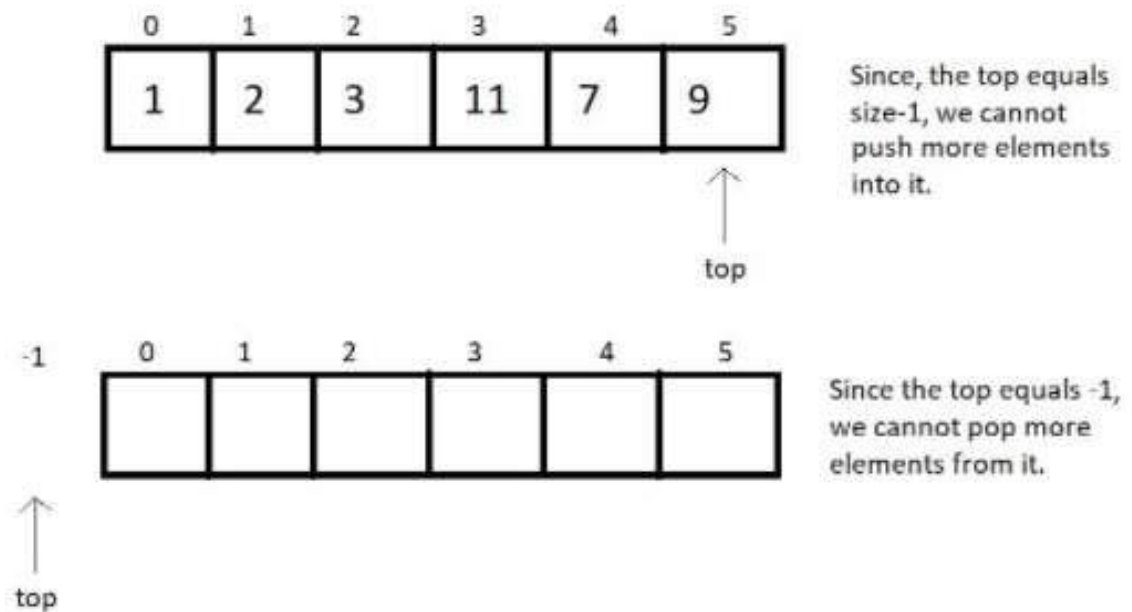
Remember

- One cannot push more elements into a full-stack.
- One cannot pop any more elements from an empty stack.

Basic Operations

- push() - to push an element into stack
- pop() - to remove the topmost element from stack
- peek (index) - to return the value at the given index\
- isempty() / isfull() - to determine whether the stack is empty or full to carry out necessary push or pop operations.

Stack is said to be in **Overflow state** when it is completely full and is said to be in **Underflow state** if it is completely empty.



Push and Pop Operation

stack abstract data type (ADT), "push" and "pop" are the fundamental operations used to manipulate the stack.

- Push operation: The push operation adds an element to the top of the stack. It takes an element as input and places it on top of the stack. The time complexity of push operation is $O(1)$.

- Pop operation: The pop operation removes and returns the topmost element from the stack. It effectively "pops" or takes out the element that was last added to the stack. pop operations would still be $O(1)$.

The push and pop operations follow the Last-In-First-Out (LIFO) principle, meaning that the most recently added element is the first one to be removed.

Stack Using Array

A stack data structure can be implemented using one dimensional array, can store only fixed number of data values. This implementation is very simple, just define a one dimensional array of specific size and insert or delete the values into that array by using LIFO principle with the help of a variable 'top'. Initially top is set to - 1 or 0.

Whenever we want to insert a value into the stack, increment the top value by one and then insert. Whenever we want to delete a value from the stack, then delete the top value and decrement the top value by one.

Stack Operations using Array

A stack can be implemented using array as follows...

steps to create an empty stack.

- Step 1: Include all the header files which are used in the program and define a constant 'SIZE' with specific value.
- Step 2: Declare all the functions used in stack implementation.
- Step 3: Create a one dimensional array with fixed size (int stack[SIZE])
- Step 4: Define a integer variable 'top' and initialize with '-1'. (int top = -1)
- Step 5: In main method display menu with list of operations and make suitable function calls to perform operation selected by the user on the stack.

push(value) - Inserting value into the stack

In a stack, push() is a function used to insert an element into the stack. In a stack, the new element is always inserted at top position. Push function takes one integer value as parameter and inserts that value into the stack. steps to push an element on to the stack...

- Step 1: Check whether stack is FULL. (top == SIZE-1)
- Step 2: If it is FULL, then display "Stack is FULL/Stack Overflow!!! Insertion is not possible!!!" and terminate the function.
- Step 3: If it is NOT FULL, then increment top value by one (top++) and set stack[top] to value (stack[top] = value).

pop() - Delete a value from the Stack

In a stack, pop() is a function used to delete an element from the stack. In a stack, the element is always deleted from top position. Pop function does not take any value as parameter.

steps to pop an element from the stack...

- Step 1: Check whether stack is EMPTY. (top == -1)
- Step 2: If it is EMPTY, then display "Stack is EMPTY!!! Deletion is not possible!!!" and terminate the function.
- Step 3: If it is NOT EMPTY, then delete stack[top] and decrement top value by one (top--).

display() - Displays the elements of a Stack

We can use the following steps to display the elements of a stack...

- Step 1: Check whether stack is EMPTY. (top == -1)
- Step 2: If it is EMPTY, then display "Stack is EMPTY!!!" and terminate the function.
- Step 3: If it is NOT EMPTY, then define a variable 'i' and initialize with top. Display stack[i] value and decrement i value by one (i--).
- Step 3: Repeat above step until i value becomes '0'.

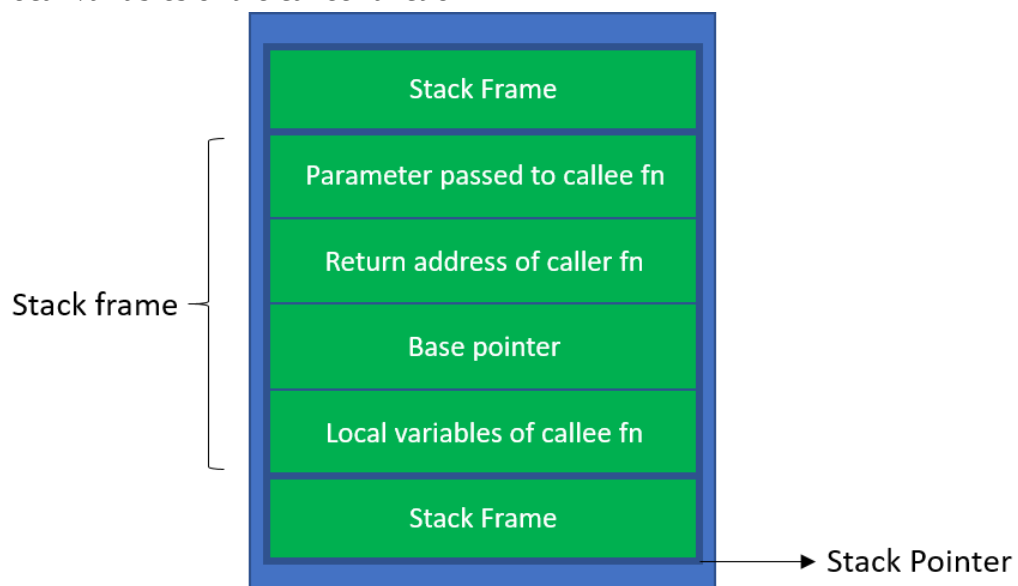
//Programs are shared in classroom

Application of Stack

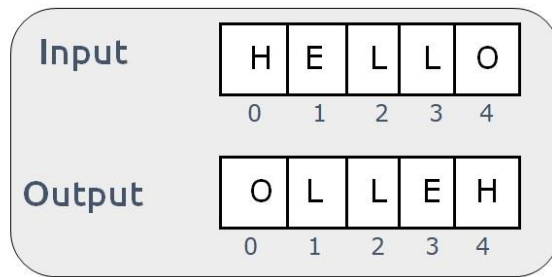
1. Stack Frame

- Each function has local memory associated with it to hold incoming parameters, local variables, and (in some cases) temporary variables. This region of memory is called a stack frame and is allocated on the process' stack.
- Each stack frame contains 4 parts:
 - Parameter passed to the callee function
 - Return address of the caller function
 - Base pointer

Local variables of the callee function



2. String reverse



Stack is a Last-In-First-Out (LIFO) data structure. If we iterate over the string and push each character from start to end, the stack will contain the string in a reverse way.

Approach:

- Take an empty stack
- Iterate over the given string from start to end.
- In each iteration, push the character at index i to the stack.
- After the first loop is completed, set a while loop till the stack is non-empty.
- Pop the character at the stack and start re-assigning the string.

Time Complexity: $O(N)$

Reason: We iterate for the length of the string two times.

Space Complexity: $O(N)$

Reason: We are using an external stack

3. Expression Evaluation given below:-

Expression Notation

Stacks provide a convenient data structure for handling the order of operations, managing operands and operators, and ensuring the correct evaluation of expressions in various expression notations.

1. Infix Notation

Infix is the day to day notation that we use of format $A + B$ type. The general form can be classified as $(a \text{ op } b)$ where a and b are operands(variables) and op is Operator.

Eg. $A+B$

2. Postfix Notation

Postfix is notation that compiler uses/converts to while reading left to right and is of format $AB+$ type. The general form can be classified as $(ab \text{ op})$ where a and b are operands (variables) and op is Operator.

Eg. $AB+$

3. Prefix Notation

Prefix is notation that compiler uses/converts to while reading right to left (some compilers can also read prefix left to right) and is of format +AB type. The general form can be classified as (op ab) where a and b are operands(variables) and op is Operator.

Eg. +AB

Precedence Level :-

Operators	Symbols
Parenthesis	(), { }, []
Exponents	^
Multiplication and Division	*, /
Addition and Subtraction	+, -

Associativity Rule:-

Operators	Associativity
^	Right to Left
*, /	Left to Right
+, -	Left to Right

Where '\$' = '^' both (power).

Expression Evaluation :Infix and Postfix

Q. Convert the post fix:-

$$((A+B)*C - (D-E) \$(F+G))$$

Evaluate post fix where A=1, B=2, C= 3, D= 4, E= 3, F=2, G=1.

Verify it by evaluating it's infix expression.

Step Symbol Operand in stack Post fix operation

Step	Symbol	Operand in stack	Post fix operation
1	(	#(	
2	(	#((	A
3	+	#((+	A
4	B	#((+ B	AB
5	)	#()	AB+
6	*	#(*	AB+
7	C	#(* C	AB+C
8	-	#(-	AB+C*
9	(	#(-(	AB+C*
10	D	#(-(D	AB+C*D
11	-	#(-(-	AB+C*D
12	E	#(-(- E	AB+C*DE
13	)	#(-	AB+C*DE-
14	)	#	AB+C*DE--
15	\$	#\$	AB+C*DE--
16	(	#\$ (	AB+C*DE--
17	F	#\$ (F	AB+C*DE - -f
18	+	#\$ (+	AB+C*DE--f
19	G	#\$ (+ G	AB+C*DE - - FG
20	)	#	AB+C*DE- - FG+ \$

The required postfix expression is AB+C*DE - - FG +\$

Now 1,2, +3, *4, 3, -, -, 2, 1, +, \$, #

Step Symbol Operand in stack

Step	Symbol	Operand in stack
1	1	1
2	2	1,2
3	+	3
4	3	3,3
5	*	9
6	4	9,4
7	3	9,4,3,1
8	-	3,1
9	-	8
10	1	8,2
11	1	8,2,1
12	\$	8,3
13	\$	24

For checking

The given infix expression is

$$((A+B)*C-(D-e))$(F+G)$$

$$= ((1+2)*3-(4-3))4(2+1)$$

$$= (3*3 -1) $3$$

$$= (9-1)$3$$

$$= 8 $ 9$$

$$= 24$$

Expression Conversion :Infix to Postfix and Postfix to Infix

Infix to Postfix

Any expression can be represented using three types of expressions (Infix, Postfix, and Prefix). We can also convert one type of expression to another type of expression like Infix to Postfix, Infix to Prefix, Postfix to Prefix and vice versa.

To convert any Infix expression into Postfix or Prefix expression we can use the following procedure...

- Find all the operators in the given Infix Expression.
- Find the order of operators evaluated according to their Operator precedence.
- Convert each operator into required type of expression (Postfix or Prefix) in the same order.

Algorithm:-

1. Begin scanning the expression from left to right.
2. If the current character being scanned is an operand, output (or append) it directly to the postfix expression.
3. If the current character is an operator:
4. If the stack is empty or the precedence of the current operator is higher than the precedence of the operator at the top of the stack, push the current operator onto the stack.
5. If the precedence of the current operator is lower than or equal to the precedence of the operator at the top of the stack, pop operators from the stack and output them until either the stack is empty, an open parenthesis is encountered, or the precedence of the operator at the top of the stack is lower than the current operator. Then, push the current operator onto the stack.
6. If the scanned character is an opening parenthesis '(', push it onto the stack.
7. If the scanned character is a closing parenthesis ')', pop operators from the stack and output them until an opening parenthesis '(' is encountered. Discard both the opening and closing parentheses.
8. Repeat steps 2 through 5 until all characters in the infix expression have been scanned.
9. Output (or append) any remaining operators in the stack to the postfix expression.
10. The resulting postfix expression is the desired output.

Example : Conversion of $A + B * C / D - F + A ^ E$ to Postfix.

Symbol	Stack	Output/Postfix	Remarks
A		A	
+	+	A	
B	+	AB	
*	+*	AB	
C	+*	ABC	
/	+/*	ABC*	* is popped
D	+/*	ABC*D	
-	-	ABC*D/+	/ and + are popped
F	-	ABC*D/+F	
+	+	ABC*D/+F-	- is popped
A	+	ABC*D/+F-A	
^	+^	ABC*D/+F-A	
E	+^	ABC*D/+F-AE	
EoE		ABC*D/+F-AE^+	End of Expression is reached and all elements are popped

Final Postfix Expression is : $ABC*D/+F-AE^+$

E.g. Question:- $((A - (B + C)) * D) \$ (E + F)$

<i>sybm</i>	<i>postfix string</i>	<i>opstk</i>
(		(
(		((
A	A	((
-	A	((-
(	A	((- (
B	AB	((- (
+	AB	((- (+
C	ABC	((- (+
)	ABC +	((-
)	ABC + -	(
*	ABC + -	(*
D	ABC + - D	(*
)	ABC + - D *	
\$	ABC + - D *	\$
(	ABC + - D *	\$(
E	ABC + - D * E	\$(
+	ABC + - D * E	\$(+
F	ABC + - D * EF	\$(+
)	ABC + - D * EF +	\$
	ABC + - D * EF + \$	

Infix to Prefix Conversion :-

- Create an empty stack and an empty output string.
- Reverse the infix expression: Reverse the order of all elements in the infix expression, including operands and operators.
- Iterate through the reversed infix expression from left to right.
- If the current character is an operand (number or variable), append it to the output string.
- If the current character is a closing bracket ')', push it onto the stack.
- If the current character is an operator or an opening bracket '(', compare its precedence with the precedence of the operator at the top of the stack.
- If the current operator has higher precedence or equal precedence than the operator at the top of the stack or the stack is empty, push the current operator onto the stack.
- If the current operator has lower than the operator at the top of the stack, pop the operators from the stack and append them to the output string until an operator with lower precedence or equal is encountered or the stack becomes empty. Then push the current operator onto the stack.
- If the current character is an opening bracket '(', pop the operators from the stack and append them to the output string until the corresponding closing bracket ')' is encountered. Discard the closing bracket.
- Repeat steps 4 to 9 until all characters in the reversed infix expression have been processed.
- Pop the remaining operators from the stack and append them to the output string.

- Reverse the output string to obtain the prefix expression.

Example : Conversion of $((a/b)+c)-(d+(e*f))$ to Prefix

Reverse of given expression : $((f*e)+d)-(c+(b/a))$

Now we have to convert the reversed expression to prefix by following the rule of infix to postfix.

Symbol	Stack	Output/Postfix	Remarks
(	(		
(	((		
f	((	f	
*	((*	f	
e	((*	fe	
)	(	fe*	* is popped
+	(+	fe*	
d	(+	fe*d	
)		fe*d+	+ is popped
-	-	fe*d+	
(	(	fe*d+	
c	(	fe*d+c	
+	(+	fe*d+c	
(	(+(	fe*d+c	
b	(+(	fe*d+cb	
/	(+(/	fe*d+cb	
a	(+(/	fe*d+cba	
)	(+	fe*d+cba/	/ is popped
)		fe*d+cba/+	+ is popped
EOE		fe*d+cba/+	End of Expression is reached

Final Postfix Expression : $fe*d+cba/+$

Reverse of Final Postfix : $+/abc+d*ef$ is the required infix expression.

//Discuss Advantages and Disadvantages of Infix , postfix and prefix over each other.